

UNIVERSIDAD PERUANA UNIÓN

FACULTAD DE FIA

E. P. INGENIERIA DE SISTEMAS



**DOCUMENTACIÓN DE ENDPOINTS, DTO Y PRUEBAS DE
CONEXIÓN — SESIÓN 07**

Autor:

WALTER JOSUÉ LLONTOP VIVAS

Docente:

Mg. Benjamín David Reyna Barreto

Lima, 26 de setiembre de 2026

Sesión 07 – Unidad 2: Conectividad, CRUD REST y persistencia multiplataforma

Repositorio: github.com/JosueLV/PharmaMobil · Rama: feature/ktor-client · Commit: 099c06a

1. Catálogo de endpoints

URL base: <https://api.escuelajs.co/api/v1/> · **Versión de API:** v1 · **Recurso principal:** products.

De los cinco endpoints documentados, `GET /products` y `GET /products/{id}` ya están implementados y probados en el cliente Ktor (Sesión 7). `POST`, `PUT` y `DELETE` se documentan como parte de la planificación del CRUD que se implementará en la Sesión 8.

Método	Ruta	Parámetros	Respuesta esperada	Códigos de error
GET	/products	limit, offset (query)	200 · arreglo de productos	500
GET	/products/{id}	id (ruta)	200 · un producto	400, 404
POST	/products	cuerpo JSON (title, price, description, categoryId, images)	201 · producto creado	400, 401
PUT	/products/{id}	id + cuerpo JSON	200 · producto actualizado	400, 404
DELETE	/products/{id}	id (ruta)	200 · confirmación (true)	401, 404

Nota de ejecución real: al probar `GET /products/{id}` con un identificador fuera de rango (999999), la API respondió con código 400 (Bad Request) en lugar de 404 — ver bitácora de pruebas, escenario 2.

2. Diccionario de DTO

DTO principal: ProductoDto (con sub-DTO CategoriaDto).

Campo JSON	Tipo Kotlin	Obligatorio	Valor por defecto	Campo en el dominio
id	Int	Sí	—	Producto.id (convertido a Long)
title	String	Sí	—	Producto.nombre
price	Double	Sí	—	Producto.precio
description	String	No	cadena vacía	No mapeado (el dominio no tiene este campo)
images	List<String>	No	lista vacía	No mapeado (el dominio no tiene este campo)
category	CategoriaDto?	No	null	No mapeado (el dominio no tiene este campo)

CategoriaDto: id (Int), name (String) — no se usa actualmente en el mapeo al dominio.

Nota sobre el mapeo: la API pública `escuelajs` es una API genérica de productos y no expone los campos `stock` ni `activo` que sí tiene el modelo de dominio `Producto` (usado para el inventario de la farmacia). Por eso el mapper les asigna un valor de referencia fijo (`stock = 20`, `activo = true`) en lugar de tomarlos del backend.

Fragmento real de la respuesta del servidor

```
{
  "id": 1,
  "title": "Classic Grey Hooded Sweatshirt",
  "price": 90,
  "description": "...",
  "images": ["https://i.imgur.com/....jpeg"],
  "category": { "id": 5, "name": "Otros" },
  "slug": "classic-grey-hooded-sweatshirt",
  "creationAt": "2026-..."
}
```

Nota: campos como `slug` y `creationAt` existen en la respuesta real pero no están declarados en `ProductoDto`; se ignoran gracias a `ignoreUnknownKeys = true` (ver `Producto 3`, escenario 5).

Código del DTO (ProductoDto.kt)

```
@Serializable
data class ProductoDto(
    val id: Int,
    val title: String,
    val price: Double,
    val description: String = "",
    val images: List<String> = emptyList(),
    @SerializedName("category") val categoria: CategoriaDto? = null
)

@Serializable
data class CategoriaDto(val id: Int, val name: String)
```

3. Bitácora de pruebas de conexión

Las cinco pruebas se ejecutaron sobre Android (emulador Pixel 10 Pro, Android 17, API 37), el 26 de setiembre de 2026.

Escenario 1 — Respuesta exitosa

Fecha y hora	26/09/2026, 12:36 a. m. (hora de Lima)
Plataforma	Android (emulador Pixel 10 Pro, API 37)
Pasos seguidos	Abrir la app en el emulador y navegar a la pantalla Productos; el ViewModel dispara GET /products?limit=10 en su bloque init.
Resultado esperado	Código 200 y lista de productos renderizada en la interfaz Compose.
Resultado observado	Respuesta 200 OK; la pantalla muestra los productos reales devueltos por la API (p. ej. “Classic Grey Hooded Sweatshirt”), mapeados al modelo Producto.
Conclusión	El cliente Ktor consume el endpoint correctamente y el estado ConProductos se renderiza sin errores.

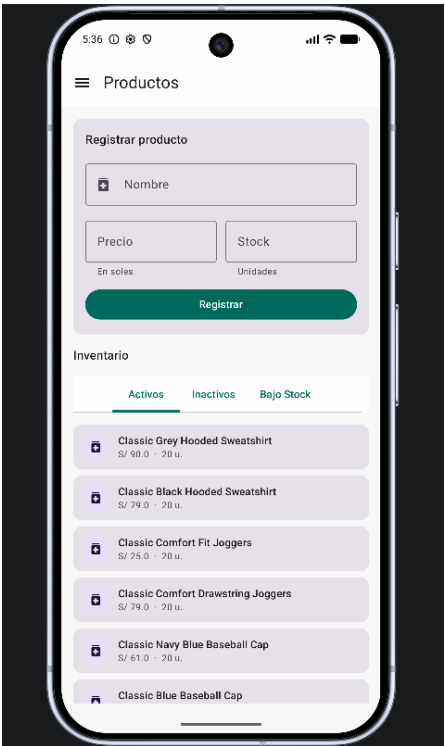


Fig. 1 — Lista de productos cargada desde la API.

Escenario 2 — Sin conexión

Fecha y hora	26/09/2026, 12:58 a. m. (hora de Lima)
Plataforma	Android (emulador Pixel 10 Pro, API 37)
Pasos seguidos	Activar el modo avión en el emulador y recargar la pantalla Productos.
Resultado esperado	Excepción de entrada/salida capturada y mensaje mostrado al usuario.
Resultado observado	Estado de error visible (“No pudimos cargar el inventario”) con el mensaje “Unable to resolve host 'api.escuelajs.co’”; la aplicación no se cierra.
Conclusión	El manejo de errores de red funciona correctamente y no interrumpe la aplicación.

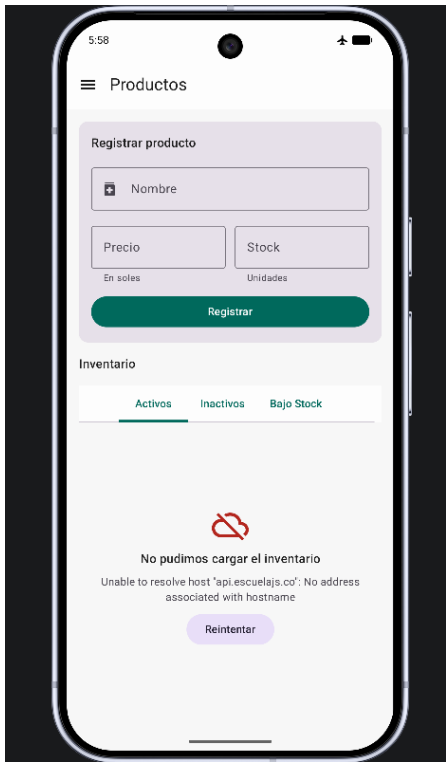


Fig. 2 — Estado de error con el emulador en modo avión.

Escenario 3 — Recurso inexistente

Fecha y hora	26/09/2026, 7:52 p. m. (hora de Lima)
Plataforma	Android (emulador Pixel 10 Pro, API 37)
Pasos seguidos	Se agregó el método <code>ProductoApi.obtenerProductoPorId(id)</code> y se invocó una vez con <code>id = 999999</code> desde un archivo de prueba temporal (<code>demo/PruebaConexion.kt</code>), revisando el resultado en Logcat.
Resultado esperado	Código 404 y <code>ClientRequestException</code> controlada.
Resultado observado	La API respondió con código 400 (Bad Request) en lugar de 404 para un id fuera de rango; la excepción <code>ClientRequestException</code> fue capturada correctamente sin cerrar la aplicación.
Conclusión	La API real usa 400 para identificadores inválidos en lugar de 404; el manejo de excepciones sigue siendo correcto y controlado.

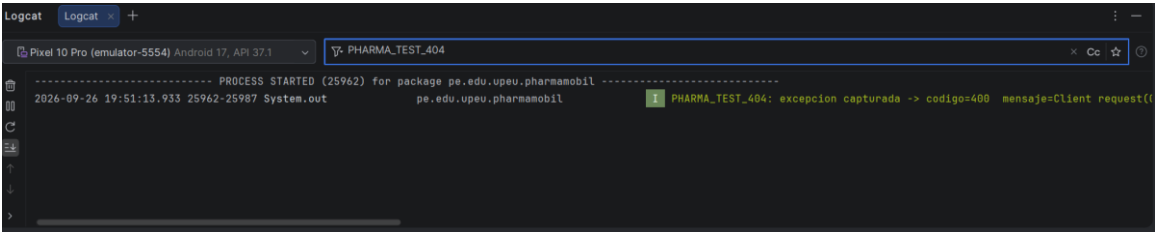


Fig. 3 — Logcat con la excepción capturada (código 400).

Escenario 4 — Tiempo de espera agotado

Fecha y hora	26/09/2026, 7:58 p. m. (hora de Lima)
Plataforma	Android (emulador Pixel 10 Pro, API 37)
Pasos seguidos	Se redujo temporalmente requestTimeoutMillis a 1 ms en HttpClientFactory y se recargó la pantalla Productos.
Resultado esperado	HttpRequestTimeoutException controlada; la interfaz no se congela.
Resultado observado	Excepción capturada en Logcat (“Request timeout has expired...”) y la interfaz mostró el estado de error de inmediato, sin quedar cargando indefinidamente.
Conclusión	El timeout configurado protege a la interfaz de quedar bloqueada ante una respuesta lenta del servidor.

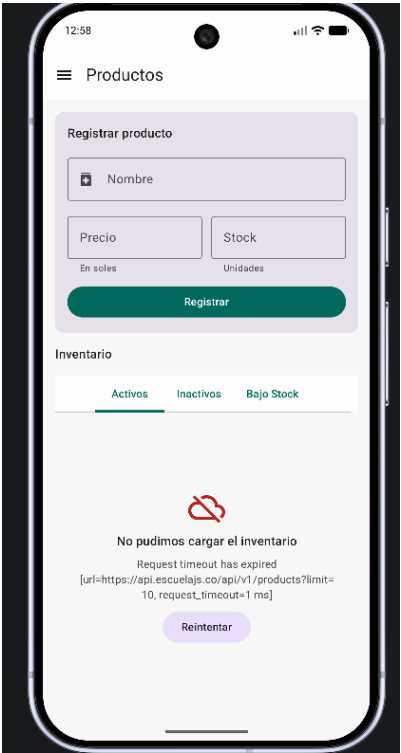


Fig. 4a — Estado de error por timeout en la interfaz.

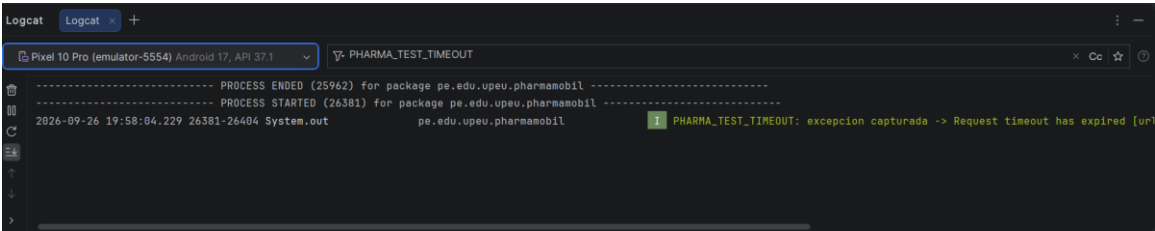


Fig. 4b — Logcat con HttpRequestTimeoutException.

Escenario 5 — Campo desconocido en el JSON

Fecha y hora	26/09/2026, 8:06 p. m. (hora de Lima)
Plataforma	Android (emulador Pixel 10 Pro, API 37)
Pasos seguidos	Se desactivó temporalmente ignoreUnknownKeys (= false) en HttpClientFactory y se recargó la pantalla Productos.
Resultado esperado	Con la opción activa la aplicación continúa; sin ella se produce SerializationException.
Resultado observado	SerializationException por el campo desconocido “slug” (“Encountered an unknown key 'slug' at path: \$[0]...”); la aplicación no se cerró y mostró el estado de error.
Conclusión	Se confirma la importancia de mantener ignoreUnknownKeys = true para tolerar cambios futuros del backend sin romper la aplicación.

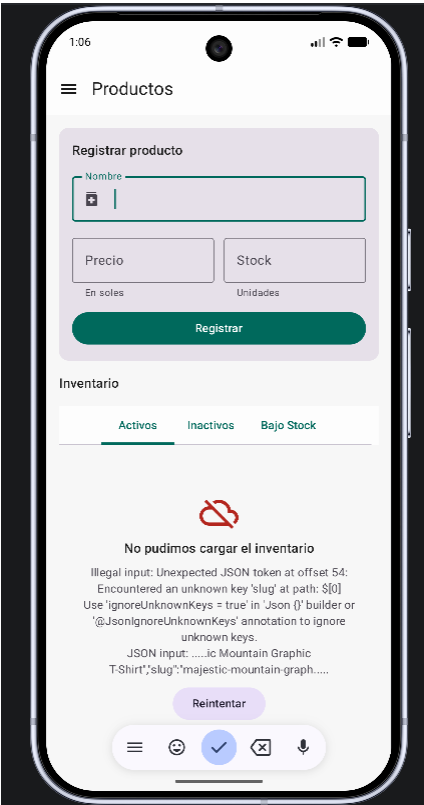


Fig. 5 — *SerializationException por campo desconocido.*

4. Evidencias y repositorio

Elemento	Detalle
Capturas de ejecución	Android: incluidas en la bitácora (Producto 3). No se incluye evidencia de iOS: el docente solicitó evidencia únicamente en Android para esta actividad.
Repositorio	github.com/JosueLV/PharmaMobil
Rama	feature/ktor-client
Commit de referencia	099c06a — “test: agregar pruebas de conexion para escenarios de error (404, timeout, JSON)”
README.md	Actualizado con la sección “Conectividad REST” (URL base, endpoint, DTO y manejo de errores).